

UniApp uni.request 网络请求：三种回调结果调用全解析

在UniApp跨平台开发中，`uni.request` 是发起HTTP/HTTPS网络请求的核心API，支持通过回调函数或Promise方式处理请求结果。其中，“回调结果”主要分为**成功回调（success）**、**失败回调（fail）**、**完成回调（complete）**三类，分别对应请求成功响应、请求异常失败、请求无论成败最终执行三种场景。本文将从基础语法、回调特性、实战示例三个维度，全面解析这三种回调的调用逻辑与使用技巧。

一、核心认知：三种回调的定位与关系

uni.request的回调机制遵循“分工明确、覆盖全流程”的设计原则，三种回调的核心定位与执行逻辑如下：

- **success（成功回调）**：仅当请求正常发送且服务器返回有效响应（状态码通常为200~299）时执行，用于处理接口返回的业务数据。
- **fail（失败回调）**：当请求发送失败（如网络中断、域名解析错误）或服务器返回异常响应（状态码 ≥ 400 ）时执行，用于捕获网络错误或服务端异常。
- **complete（完成回调）**：无论请求成功还是失败，最终都会执行，用于释放请求过程中的资源（如关闭加载动画）、统一处理后续逻辑。



执行顺序规律：成功时依次执行 success → complete；失败时依次执行 fail → complete；complete 是请求生命周期的“终点回调”，优先级低于前两者但必定执行。

二、基础语法：三种回调的调用形式

uni.request支持“对象参数配置”和“Promise链式调用”两种写法，三种回调在不同写法中的调用形式存在差异，核心区别在于是否依赖回调函数参数。

1. 写法1：对象参数配置（传统回调式）

通过在uni.request的配置对象中直接定义success、fail、complete三个回调函数，是最基础、最常用的写法，适合简单场景。

```

1 uni.request({
2   url: 'https://api.example.com/user/info', // 接口地址 (必选)
3   method: 'GET', // 请求方法 (默认GET)
4   data: { userId: 123 }, // 请求参数
5   // 1. 成功回调: 服务器正常响应
6   success: (res) => {
7     console.log('请求成功, 响应数据: ', res)
8     // 业务逻辑: 解析数据、更新页面等
9   },
10  // 2. 失败回调: 请求发送失败或响应异常
11  fail: (err) => {
12    console.error('请求失败, 错误信息: ', err)
13    // 业务逻辑: 提示网络错误、重试等
14  },
15  // 3. 完成回调: 请求无论成败都执行
16  complete: () => {
17    console.log('请求执行完成 (成功/失败均触发) ')
18    // 业务逻辑: 关闭加载动画、释放资源等
19  }
20 })

```

2. 写法2: Promise链式调用 (异步流式)

uni.request本身返回Promise对象, 可通过 `.then()` 和 `.catch()` 链式调用替代传统回调, 其中:

- `.then(res => {})`: 对应传统写法的 success 回调, 接收成功响应数据。
- `.catch(err => {})`: 对应传统写法的 fail 回调, 捕获请求失败错误。
- `.finally(() => {})`: 对应传统写法的 complete 回调, 请求完成后执行。

Code block

```

1 uni.request({
2   url: 'https://api.example.com/goods/list',
3   method: 'POST',
4   data: { categoryId: 5 }
5 })
6 .then((res) => {
7   // 成功回调: 对应 success
8   console.log('请求成功: ', res)
9 })
10 .catch((err) => {
11   // 失败回调: 对应 fail
12   console.error('请求失败: ', err)
13 })
14 .finally(() => {

```

```
15    // 完成回调: 对应 complete
16    console.log('请求执行完成')
17  })
```

3. 写法3: async/await 语法（同步化写法）

结合ES6的 `async/await` 语法，可将异步请求转化为“同步”代码风格，更符合线性编程逻辑，三种回调通过 `try/catch/finally` 结构实现：

- `try` 代码块：对应 success 回调，执行请求成功后的逻辑。
- `catch` 代码块：对应 fail 回调，捕获请求失败的错误。
- `finally` 代码块：对应 complete 回调，请求完成后统一执行。

Code block

```
1  // 需在async函数中使用
2  const fetchGoodsData = async () => {
3    // 显示加载动画（请求前执行）
4    uni.showLoading({ title: '加载中...' })
5
6    try {
7      // 成功回调：请求成功时进入
8      const res = await uni.request({
9        url: 'https://api.example.com/goods/list',
10       data: { categoryId: 5 }
11      })
12      console.log('请求成功: ', res)
13      // 业务逻辑：解析数据
14    } catch (err) {
15      // 失败回调：请求失败时进入
16      console.error('请求失败: ', err)
17      uni.showToast({ title: '请求失败', icon: 'error' })
18    } finally {
19      // 完成回调：无论成败都进入
20      uni.hideLoading() // 关闭加载动画
21      console.log('请求执行完成')
22    }
23  }
24
25  // 调用请求函数
26  fetchGoodsData()
```

三、回调参数详解：成功与失败的核心数据

三种回调的核心价值在于获取请求过程中的关键数据，其中 success 和 fail 的参数结构差异较大，需精准解析以实现业务逻辑。

1. success 回调参数（res）结构

success 回调的参数 res 包含“请求响应头”“响应状态码”“业务数据”等核心信息，完整结构如下：

Code block

```
1  success: (res) => {
2    // 1. 响应状态码 (200表示成功)
3    console.log('状态码: ', res.statusCode)
4    // 2. 响应头信息 (如Content-Type、Set-Cookie)
5    console.log('响应头: ', res.header)
6    // 3. 服务器返回的业务数据 (核心)
7    console.log('业务数据: ', res.data)
8    // 4. 请求配置信息 (请求时的url、method等)
9    console.log('请求配置: ', res.config)
10 }
```



注意：statusCode 为 HTTP 响应状态码（如200、404、500），而 res.data.code 通常是业务状态码（由后端定义，如0表示业务成功，1表示参数错误），需区分判断。

2. fail 回调参数（err）结构

fail 回调的参数 err 包含“错误码”“错误信息”“请求配置”等信息，用于定位请求失败原因，完整结构如下：

Code block

```
1  fail: (err) => {
2    // 1. 错误码 (UniApp定义的错误类型标识)
3    console.log('错误码: ', err.errCode)
4    // 2. 错误描述信息 (如“网络错误”“域名解析失败”)
5    console.log('错误信息: ', err.errMsg)
6    // 3. 响应状态码 (若服务器返回异常状态码，如404、500)
7    console.log('响应状态码: ', err.statusCode)
8    // 4. 请求配置信息
9    console.log('请求配置: ', err.config)
10 }
```

常见错误码及含义：

错误码	错误类型	常见原因
1001	网络错误	设备无网络、网络中断
1003	域名解析失败	接口地址错误、DNS配置问题
1004	连接超时	服务器响应慢、网络延迟高
404	资源不存在	接口路径错误
500	服务器内部错误	后端代码异常

3. complete 回调参数

complete 回调的参数与 success 或 fail 一致：若请求成功，参数为 res（同 success）；若请求失败，参数为 err（同 fail）。其核心作用是“统一收尾”，与业务逻辑解耦。

Code block

```
1  complete: (resOrErr) => {
2    // 关闭加载动画（无论成败都执行）
3    uni.hideLoading()
4
5    // 可根据参数判断请求结果（可选）
6    if (resOrErr.statusCode === 200) {
7      console.log('请求成功后的收尾')
8    } else {
9      console.log('请求失败后的收尾')
10   }
11 }
```

四、实战场景：三种回调的典型应用

结合实际业务需求，三种回调的配合使用可实现“请求状态反馈、数据处理、异常兜底”的完整流程，以下是高频场景示例。

1. 场景1：基础数据请求（带加载动画）

需求：请求商品列表数据，请求前显示加载动画，成功后渲染数据，失败后提示错误，完成后关闭加载动画。

Code block

```
1  <template>
2    <view class="goods-page">
3      <view class="goods-item" v-for="item in goodsList" :key="item.id">
4        <text class="goods-name">{{ item.name }}</text>
5        <text class="goods-price">¥{{ item.price }}</text>
6      </view>
7    </view>
8  </template>
9
10 <script setup>
11 import { ref } from 'vue'
12
13 const goodsList = ref([])
14
15 // 发起商品列表请求
16 const fetchGoodsList = () => {
17   // 1. 请求前：显示加载动画
18   uni.showLoading({ title: '加载商品中...' })
19
20   uni.request({
21     url: 'https://api.example.com/goods/list',
22     method: 'GET',
23     data: { categoryId: 1 },
24     // 2. 成功回调：解析数据并渲染
25     success: (res) => {
26       // 区分HTTP状态码和业务状态码
27       if (res.statusCode === 200 && res.data.code === 0) {
28         goodsList.value = res.data.data
29       } else {
30         // 业务异常（如参数错误），也提示错误
31         uni.showToast({ title: res.data.msg || '数据获取失败', icon: 'error' })
32       }
33     },
34     // 3. 失败回调：提示网络或服务端错误
35     fail: (err) => {
36       let errMsg = '请求失败，请重试'
37       if (err.errCode === 1001) errMsg = '无网络，请检查网络设置'
38       if (err.statusCode === 404) errMsg = '接口不存在'
39       uni.showToast({ title: errMsg, icon: 'error' })
40     },
41     // 4. 完成回调：关闭加载动画
42     complete: () => {
43       uni.hideLoading()
44     }
45   })
46 }
47
```

```
48 // 页面加载时发起请求
49 fetchGoodsList()
50 </script>
```

2. 场景2：用户登录（带参数验证与token存储）

需求：用户提交登录表单，请求前验证参数，成功后存储token并跳转到首页，失败后提示具体错误，完成后重置按钮状态。

Code block

```
1  <template>
2    <view class="login-page">
3      <input v-model="username" placeholder="请输入用户名" />
4      <input v-model="password" type="password" placeholder="请输入密码" />
5      <button @click="handleLogin" :disabled="isLoading">
6        {{ isLoading ? '登录中...' : '登录' }}
7      </button>
8    </view>
9  </template>
10
11 <script setup>
12 import { ref } from 'vue'
13
14 const username = ref('')
15 const password = ref('')
16 const isLoading = ref(false) // 按钮加载状态
17
18 const handleLogin = async () => {
19   // 1. 请求前：参数验证
20   if (!username.value.trim() || !password.value.trim()) {
21     uni.showToast({ title: '用户名或密码不能为空', icon: 'none' })
22     return
23   }
24
25   // 2. 请求前：设置按钮加载状态
26   isLoading.value = true
27
28   try {
29     // 3. 成功回调：登录成功处理
30     const res = await uni.request({
31       url: 'https://api.example.com/user/login',
32       method: 'POST',
33       data: { username: username.value, password: password.value }
34     })
35
```

```

36     if (res.statusCode === 200 && res.data.code === 0) {
37         // 存储token (用于后续接口请求)
38         uni.setStorageSync('token', res.data.data.token)
39         uni.showToast({ title: '登录成功' })
40         // 跳转到首页
41         uni.switchTab({ url: '/pages/home/home' })
42     } else {
43         // 业务错误 (如用户名密码错误)
44         uni.showToast({ title: res.data.msg, icon: 'error' })
45     }
46 } catch (err) {
47     // 4. 失败回调: 网络或服务端错误
48     uni.showToast({ title: '登录异常, 请重试', icon: 'error' })
49     console.error('登录请求失败: ', err)
50 } finally {
51     // 5. 完成回调: 重置按钮状态
52     isLoading.value = false
53 }
54 }
55 </script>

```

3. 场景3：文件上传（带进度反馈与异常处理）

需求：上传图片文件，请求过程中显示进度条，成功后获取图片URL，失败后提示错误，完成后隐藏进度条。（注：文件上传用uni.uploadFile，但回调逻辑与uni.request一致）

Code block

```

1  <template>
2    <view class="upload-page">
3      <button @click="chooseImage">选择图片</button>
4      <progress v-if="showProgress" :percent="uploadPercent" stroke-width="4" />
5      <image v-if="imageUrl" :src="imageUrl" mode="widthFix" class="uploaded-
  img" />
6    </view>
7  </template>
8
9  <script setup>
10  import { ref } from 'vue'
11
12  const imageUrl = ref('')
13  const showProgress = ref(false)
14  const uploadPercent = ref(0)
15
16  // 选择图片
17  const chooseImage = () => {

```



```
18 uni.chooseImage({
19   count: 1,
20   success: (chooseRes) => {
21     // 选择图片后发起上传请求
22     uploadImage(chooseRes.tempFilePaths[0])
23   }
24 })
25 }
26
27 // 上传图片
28 const uploadImage = (tempFilePath) => {
29   showProgress.value = true
30
31   uni.uploadFile({
32     url: 'https://api.example.com/file/upload',
33     filePath: tempFilePath,
34     name: 'file', // 后端接收文件的字段名
35     header: {
36       'Authorization': `Bearer ${uni.getStorageSync('token')}` // 携带token
37     },
38     // 进度回调（额外功能，非核心回调）
39     progress: (progressRes) => {
40       uploadPercent.value = Math.floor(progressRes.progress)
41     },
42     // 成功回调
43     success: (res) => {
44       const data = JSON.parse(res.data) // 上传接口返回的是字符串，需解析
45       if (res.statusCode === 200 && data.code === 0) {
46         imageUrl.value = data.data.url // 显示上传后的图片
47         uni.showToast({ title: '上传成功' })
48       } else {
49         uni.showToast({ title: data.msg || '上传失败', icon: 'error' })
50       }
51     },
52     // 失败回调
53     fail: (err) => {
54       uni.showToast({ title: '上传异常，请重试', icon: 'error' })
55       console.error('图片上传失败：', err)
56     },
57     // 完成回调
58     complete: () => {
59       showProgress.value = false // 隐藏进度条
60       uploadPercent.value = 0 // 重置进度
61     }
62   })
63 }
64 </script>
```

五、避坑要点：回调使用的常见问题与解决方案

1. 问题1：success回调未执行，但请求实际成功？

原因：1. 后端返回的Content-Type与数据格式不匹配（如声明application/json但返回字符串）；2. 跨域问题导致浏览器拦截响应；3. 业务状态码判断错误（将res.data.code≠0误认为请求失败）。

解决方案：

检查response header的Content-Type，确保与后端返回数据格式一致。UniApp中配置manifest.json的“h5”→“devServer”解决跨域问题（开发环境）。success回调中先判断HTTP状态码（res.statusCode），再判断业务状态码（res.data.code）。

2. 问题2：fail回调执行，但不知道具体失败原因？

原因：未解析fail回调的err参数，仅依赖默认错误信息，无法定位具体问题。

解决方案：打印err的完整信息，重点关注errCode、statusCode、errMsg三个字段：

Code block

```
1  fail: (err) => {
2    console.error('请求失败详情: ', {
3      errCode: err.errCode,
4      statusCode: err.statusCode,
5      errMsg: err.errMsg,
6      config: err.config
7    })
8    // 根据errCode给出具体提示
9    if (err.errCode === 1004) {
10     uni.showToast({ title: '请求超时, 请检查网络', icon: 'error' })
11   } else if (err.statusCode === 500) {
12     uni.showToast({ title: '服务器维护中', icon: 'error' })
13   }
14 }
```

3. 问题3：complete回调中重复执行业务逻辑？

原因：将数据处理、错误提示等业务逻辑放在complete中，与success/fail重复，导致逻辑混乱。

解决方案：complete仅用于“通用收尾逻辑”，如关闭加载动画、重置状态，不处理业务数据或错误提示：

Code block

```
1  // 错误写法：complete中处理业务逻辑
```

```
2  complete: (res) => {
3    if (res.statusCode === 200) {
4      goodsList.value = res.data.data // 业务逻辑，应放在success中
5    }
6  }
7
8  // 正确写法: complete仅做通用收尾
9  complete: () => {
10    uni.hideLoading() // 仅关闭加载动画
11  }
```

4. 问题4: async/await写法中, fail回调不执行?

原因: async/await写法中, uni.request的失败信息通过catch捕获, 而非fail回调, 定义fail回调会失效。

解决方案: async/await写法中, 统一通过try/catch处理失败, 不定义fail回调:

Code block

```
1  // 错误写法: async/await中定义fail回调 (无效)
2  const res = await uni.request({
3    url: 'xxx',
4    fail: (err) => {
5      console.error(err) // 不会执行
6    }
7  })
8
9  // 正确写法: 通过catch捕获失败
10 try {
11   const res = await uni.request({ url: 'xxx' })
12 } catch (err) {
13   console.error(err) // 捕获失败信息
14 }
```

六、总结

uni.request的三种回调结果调用是实现网络请求全流程控制的核心, 其使用需遵循“分工明确、场景匹配”的原则:

1. **success:** 聚焦“数据处理”, 仅处理请求成功后的业务逻辑, 需区分HTTP状态码与业务状态码。
2. **fail:** 聚焦“异常兜底”, 捕获网络错误、服务端异常, 通过err参数精准定位问题并给出友好提示。

3. **complete**: 聚焦“通用收尾”，统一处理加载动画关闭、状态重置等与业务无关的逻辑，确保请求流程闭环。

实际开发中，建议根据场景选择合适的写法：简单场景用传统回调式，复杂异步流程用 `async/await+try/catch/finally`。同时，重视回调参数的解析与错误分类处理，可大幅提升网络请求的稳定性与用户体验。

（注：文档部分内容可能由 AI 生成）